

Three vertical lines of varying heights and colors (yellow, orange, and blue) are positioned on the left side of the slide.

3. Evaluación del *software*

Introducción. Revisiones

Miguel Ángel Latre
Javier Nogueras Iso

Temario de la asignatura

1. El proceso de pruebas
2. Técnicas y herramientas de pruebas o *testing*
 - ❖ Introducción
 - ❖ Técnicas generales
 - ❖ Pruebas unitarias. Herramientas
 - ❖ Pruebas de integración. Técnicas y herramientas
 - ❖ Depuración
3. Técnicas y herramientas de evaluación estática
 - ❖ **Revisiones**
 - ❖ Análisis estático
 - Técnicas
 - Herramientas
4. Evaluación y *testing* de interfaces persona ordenador

Temario (técnicas y herramientas)

	Técnicas	Herramientas
Técnicas generales basadas en la estructura	Puntos de decisión Camino con profundidad	Medición de cobertura
Técnicas generales basadas en la especificación	Particiones equivalencia Pruebas de pares / tablas ortogonales	Pict 3.3 AllPairs
Pruebas de desarrollo de unidad	Generales	Aserciones; JUnit 4; JUnit 5/6
de integración	Sustitutos	Mockito
Pruebas de sistema y aceptación (con GUI)	Generales	Selenium Espresso, UI Automator Cucumber
Evaluación (documentación)	Inspecciones Listas de comprobación	
Análisis estático (código)	Patrones de errores Verificación formal	Inspector de IntelliJ / AS JML

Índice

- Introducción a la evaluación del *software*
- Técnicas para la evaluación del *software* («documentación»)
 - ❖ Inspecciones
 - ❖ Listas de comprobación

Evaluación del *software*

□ Definición

❖ Valoración de los productos intermedios del proceso de desarrollo

- Documento de requisitos
- Análisis
- Diseño
- Código
- Plan de proyecto
- Plan de pruebas
- Manuales de usuario y de instalación
- ...

Evaluación del *software*

❑ Objetivos

- ❖ Encontrar y corregir defectos en los productos intermedios
- ❖ Mejorar los productos
- ❖ Evitar la propagación de los defectos a otros productos intermedios posteriores o a la solución final

❑ Más efectiva que el *testing* dinámico

- ❖ Mayor número de defectos encontrados
- ❖ Menor coste/esfuerzo de corrección de los defectos encontrados

Técnicas

□ Evaluación de documentación

❖ Reuniones formales

- Inspecciones
- Revisiones
- *Walkthroughs*

❖ Listas de comprobación

□ Análisis estático del código

❖ Patrones de errores

- Referencias nulas
- Excepciones no capturadas
- Uso de memoria

❖ Verificación formal

Técnicas

□ Evaluación de documentación

❖ Reuniones formales

- Inspecciones
- Revisiones
- *Walkthroughs*

❖ Listas de comprobación

□ Análisis estático del código

❖ Patrones de errores

- Referencias nulas
- Excepciones no capturadas
- Uso de memoria

❖ Verificación formal

Inspecciones

- ❑ Reuniones (físicas o remotas) para la revisión por pares de cualquier resultado de una fase de desarrollo de *software* por personas capacitadas que buscan defectos mediante un proceso bien definido
- ❑ Conocida también como «Inspección Fagan»
- ❑ Productos susceptibles de revisión:
 - ❖ Requisitos, resultados del análisis y el diseño
 - No se pueden «ejecutar» a través de pruebas.
 - ❖ Código
 - Facilidad de mantenimiento, evolución y reusabilidad
 - Escalabilidad, eficiencia
 - Seguridad, integridad

Inspecciones

❏ Procedimiento

❖ Planificación

- Definir fecha y lugar, determinar el equipo y asignar roles, distribuir la documentación a inspeccionar, ...

❖ Lanzamiento

- Reunión opcional previa a la inspección
- Descripción del procediendo a inspectores noveles
- Explicaciones previas de la documentación por parte del autor

❖ Preparación

- Inspectores revisan la documentación, buscando defectos y anotándolos

❖ Ejecución

- Reunión de inspección

Inspecciones

□ Participantes

❖ Entre 3 y 7

- Grupos menores para inspecciones de código; mayores para otros documentos

❖ Papeles

- Moderador
- Autor
- Secretario
- Lector
- Inspectores

Inspecciones

❏ Proceso

- ❖ **Lector** describe segmento a segmento el elemento inspeccionado.
- ❖ Los **inspectores** responden indicando defectos, haciendo preguntas o sugiriendo mejoras.
 - Técnica adicional: listas de comprobación
- ❖ El **secretario** anota cada defecto, pregunta o mejora.
- ❖ El **moderador** garantiza la continuidad de la reunión:
 - Evitando centrarse en la solución de los problemas detectados,
 - (Detiene los debates sobre soluciones tras 1 o 3 minutos)
 - Fomentando la participación.

❏ Reuniones de inspección de no más de dos horas

Inspecciones

□ Resultados

❖ Evaluación del producto

- Aceptación (cambios menores, no es necesaria una nueva inspección)
- Aceptación condicionada (cambios menores)
- Cambios mayores y una nueva inspección
- Inspección no completada

❖ Realimentación sobre el propio proceso de inspección

Inspecciones

□ Beneficios

- ❖ Detección pronta de defectos en el proceso de desarrollo
 - Reducción del coste de corrección
- ❖ Reducción del coste de realización de pruebas
- ❖ Obtención de nuevas perspectivas sobre el elemento inspeccionado
- ❖ Compartición del conocimiento
 - Relativo al diseño o al *software*
 - Relativo a prácticas para la detección de defectos

Inspecciones

□ Beneficios

AETNA Life and Casualty. 4439 LOC [2]	—0 Defects in use. —25 percent reduction in development resource.
IBM RESPOND, U.K. 6271 LOC [3]	—0 Defects in use. —9 percent reduction in cost compared to walkthrus.
Standard Bank of South Af- rica. 143 000 LOC [4]	—0.15 Defects/KLOC in use. —95 percent reduction in corrective maintenance cost.
American Express, System code). 13 000 LOC	—0.3 Defects in use.

Fuente: M. Fagan. «Advances in Software Inspections». *IEEE Transactions on Software Engineering*. Vol: SE-12 (7). 1986.

Inspecciones

❑ Aspectos sociales y psicológicos

❖ Deben planificarse.

- En caso contrario, pueden verse como una intrusión o como una pérdida de tiempo.
- Debe reconocerse que aceleran el desarrollo reduciendo otro tipo de actividades de verificación.

❖ Suponen un reto:

- Los autores invierten autoestima en el producto.
- Alienta una mayor calidad para evitar que otros encuentren errores.

❖ Los autores

- Deben reconocer el valor de la retroalimentación que reciben.
- No deben entender las críticas a su código como personales.

❖ Los revisores

- Deben evitar mostrarse pedantes.
- Deben ser sensibles.

Inspecciones

❏ Problemas

- ❖ Permitir que los revisores lideren el proceso de calidad
 - “¿Qué más da? Ya lo encontrarán los revisores”
 - **La responsabilidad es siempre del autor**, no de los revisores
- ❖ Perfeccionismo antes de la inspección
 - Evitar las inspecciones
 - Someter por partes
 - Ensayar las inspecciones (consumo de tiempo)
 - Evitar la aceptación de los cambios
- ❖ Utilizar las inspecciones como evaluación de la productividad personal
- ❖ Evitar señalar defectos en las revisiones por falso compañerismo

Tipos de reuniones de evaluación

❑ Inspecciones

- ❖ Más formales
- ❖ Producto revisado terminado al 100%
- ❖ Preparación previa
- ❖ Liderada por un moderador, no por el autor
- ❖ Sigue un proceso claramente definido
- ❖ Genera un documento o acta
- ❖ Más costosa, más eficaz (encuentra más defectos)

❑ Revisiones técnicas

- ❖ Producto revisado acabado en un 60-80%

❑ Ensayos (*Walkthroughs*)

- ❖ Menos formales
- ❖ Producto revisado terminado al 50-100%
- ❖ Habitualmente, sin preparación previa
- ❖ A veces, preparación para una inspecciones formal
- ❖ Menos costosa, menos eficaz

Técnicas relacionadas con las revisiones

□ Programación en pareja

❖ Papeles

- Controlador o conductor: escribe el código.
- Observador o navegador: revisa cada línea de código, mantiene una visión global y se anticipa a los problemas.

❖ Beneficios

- Reducción del tiempo de programación
- Aumento de la calidad del código producido

❖ Desventajas

- Aumento del coste de personal (frente a la programación individual)

□ Diseño en pajera

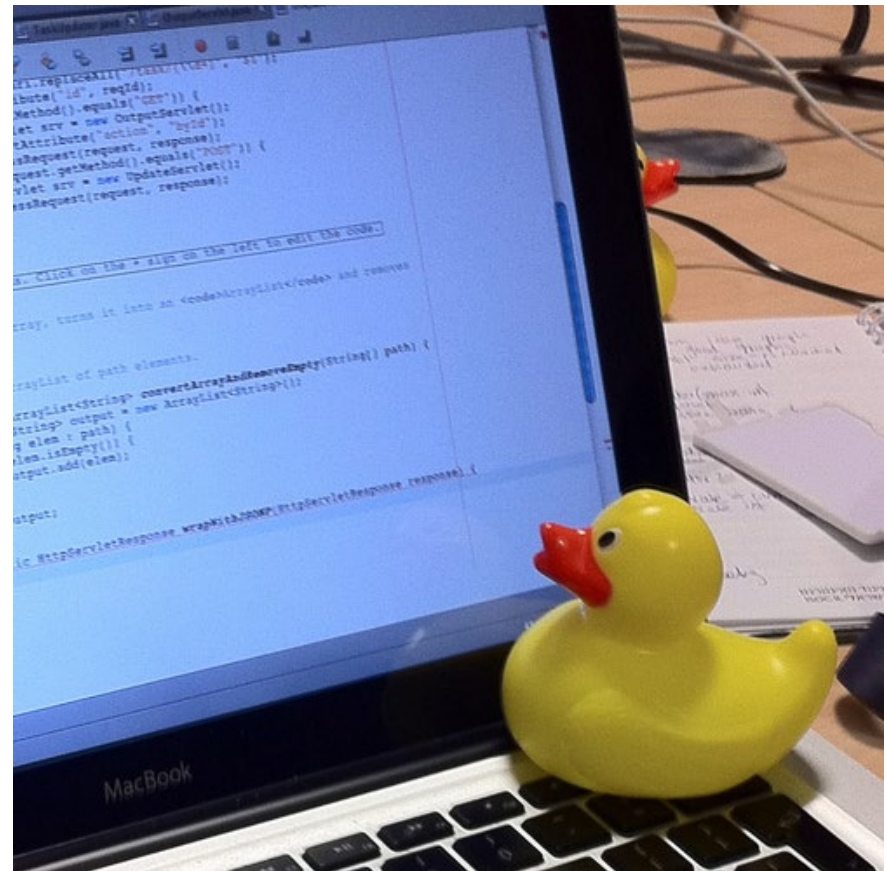
Fuente: Lisamarie Babik, «Look! It's paired programming!», *Wikimedia Commons, the free media repository*, 2007,
https://en.wikipedia.org/wiki/File:Pair_programming_1.jpg
[accedido el 24-3-2026]



Técnicas relacionadas con las revisiones

□ Revisión con patito de goma

❖ *Rubber duck debugging; rubber ducking*



Fuente: Tom Morris, «Rubber duck assisting with debugging.jpg», *Wikimedia Commons, the free media repository*, 2011,

https://commons.wikimedia.org/w/index.php?title=File:Rubber_duck_assisting_with_debugging.jpg&oldid=156498927

[accedido el 24-3-2026]

Técnicas

□ Evaluación de documentación

❖ Reuniones formales

- Inspecciones
- Revisiones
- *Walkthroughs*

❖ Listas de comprobación

□ Análisis estático del código

❖ Patrones de errores

- Referencias nulas
- Excepciones no capturadas
- Uso de memoria

❖ Verificación formal

Listas de comprobación

- Lista de preguntas o elementos para revisar un producto (documento o diagrama) relativo al análisis, diseño o código
 - ❖ Permiten reflexionar sobre algunos aspectos de un producto.
 - ❖ Deben centrarse en fuentes de error habituales.
- Objetivos
 - ❖ Problemas prioritarios
 - ❖ Problemas difíciles de encontrar con otras técnicas
 - ❖ Problemas históricos
 - ❖ Problemas específicos de un tipo de documento

Listas de comprobación

- ❑ O la lista de comprobación o el documento a revisar deben ser cortos
 - ❖ Unos siete elementos
 - ❖ Con listas más largas para documentos largos, hay que hacer subgrupos y varias revisiones
- ❑ Deben refinarse basándose en la experiencia
- ❑ Peligro: centrarse en aspectos secundarios o de estilo

Listas de comprobación. Requisitos

☐ Completitud

- ❖ Los requisitos cubren todas las necesidades establecidas por el usuario o el cliente.

☐ Validez y corrección

- ❖ Los requisitos están libres de errores y representan algo requerido por el usuario o el cliente.

☐ Precisión, no ambigüedad y claridad

- ❖ El requisito es exacto y no resulta vago, tiene una única interpretación, el significado es claro y es fácil de leer.

☐ Consistencia

- ❖ Los requisitos no se contradicen.

Listas de comprobación. Requisitos

☐ Realismo

- ❖ ¿El requisito es realizable?

☐ Trazabilidad

- ❖ ¿Se puede seguir la pista del requisito a lo largo del desarrollo?

☐ Verificabilidad

- ❖ Debe ser posible comprobar que el requisito se ha satisfecho

☐ Libres de detalles indeseables de diseño

☐ Manejabilidad (adaptabilidad)

- ❖ El requisito se puede cambiar sin afectar excesivamente al resto

Listas de comprobación. Requisitos

Ejemplo

❑ Parte de los requisitos de la aplicación Notas

- ❖ Permitir la eliminación de categorías
- ❖ Permitir asignar a cada nota una categoría previamente creada
- ❖ Permitir modificar la categoría previamente asignada a una nota
- ❖ Permitir eliminar la categoría previamente asignada a una nota

❑ Lista de comprobación para requisitos

- ❖ Completitud
- ❖ Validez, corrección
- ❖ Precisión, no ambigüedad y claridad
- ❖ Consistencia
- ❖ Realismo
- ❖ Trazabilidad
- ❖ Verificabilidad
- ❖ Libres de detalles de diseño

Listas de comprobación. Casos de uso

- ❑ Cada requisito funcional y no funcional del catálogo de requisitos se debería corresponder con algún caso de uso o aparecer reflejado en su descripción textual
- ❑ Cada caso de uso tiene asociados, al menos, un diagrama de secuencia de análisis y un diagrama de secuencia de diseño
- ❑ Cada caso de uso tiene un significado concreto en el sistema, no está repetido y se refiere a actividades informáticas y no humanas.
- ❑ Cada caso de uso proporciona un servicio real y completo al actor con el que está asociado.
- ❑ Cada caso de uso representa una interacción atómica con el actor
- ❑ ...

Listas de comprobación

□ Listas de comprobación para diagramas de

- ❖ clases
- ❖ secuencia
- ❖ paquetes
- ❖ componentes
- ❖ despliegue

Tarea de evaluación estática de la documentación de análisis y diseño de la práctica 2

Listas de comprobación

❑ Código

- ❖ ¿Los índices de los vectores u otras colecciones están en los rangos correctos?
- ❖ ¿Se ha comprobado la aritmética entera, especialmente la división entera, para evitar redondeos o truncamientos no esperados?
- ❖ ¿Se cierran los ficheros y otro tipo de recursos, incluso en situaciones de error?
- ❖ ¿Se inicializan todas las variables, referencias y punteros antes de usarse?
- ❖ ¿En una instrucción **switch** de C, C++, C#, Java, ..., todos los casos terminan con **break** o **return**?
- ❖ ¿Los objetos se comparan con `equals` y no con `==`?
- ❖ ...

❑ Estilo

- ❖ ¿Los identificadores utilizados son descriptivos y siguen las normas fijadas?
- ❖ ¿Hay constantes literales que sean susceptibles de ser constantes simbólicas?
- ❖ ...

